November 14th, 2025





Computer Vision

BSc AIDAMS ESSEC/CentraleSupélec
IDSI51006 - Winter 2025
Louise Davy

Outline for today

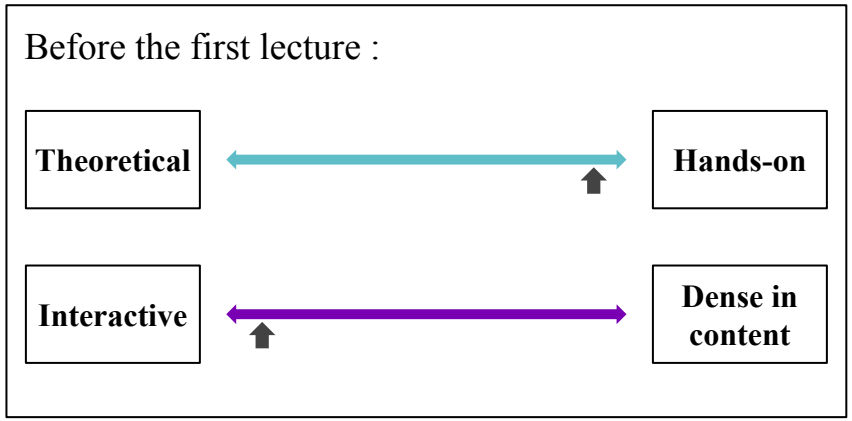
01. Recap of last session
02. A brief history on deep learning for images
03. Our first perceptron
04. Our first multi-layer perceptron
05. Our first convolutional neural network
06. Quiz

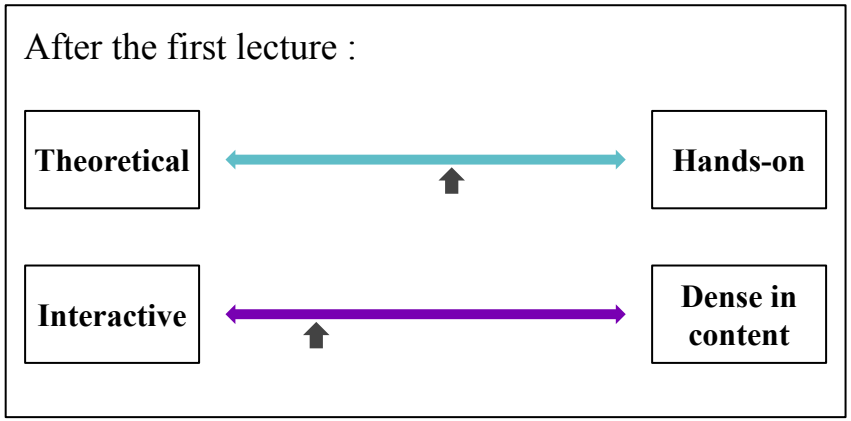
Outline for today

01. **Recap of last session**
02. A brief history on deep learning for images
03. Our first perceptron
04. Our first multi-layer perceptron
05. Our first convolutional neural network
06. Quiz

Time for the recap !

My recap





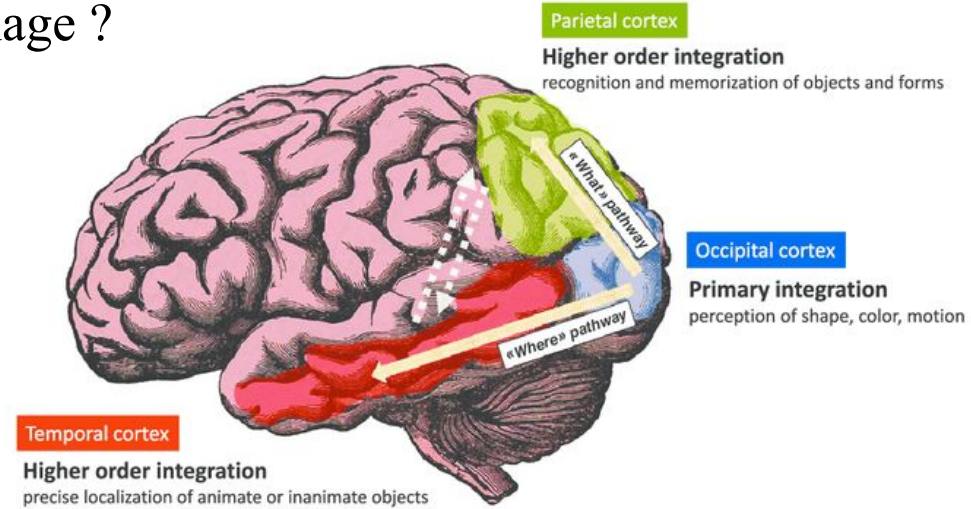
→ Adding a bit more formalism and depth under the same time constraint meant either reducing content or limiting interactive activities. I chose the latter.

Outline for today

01. Recap of last session
02. A brief history on deep learning for images
03. Our first perceptron
04. Our first multi-layer perceptron
05. Our first convolutional neural network
06. Quiz

A brief history of deep learning

Remember this image ?



Source : Clonal selection versus clonal cooperation: the integrated perception of immune objects, Serge Nataf (2016)

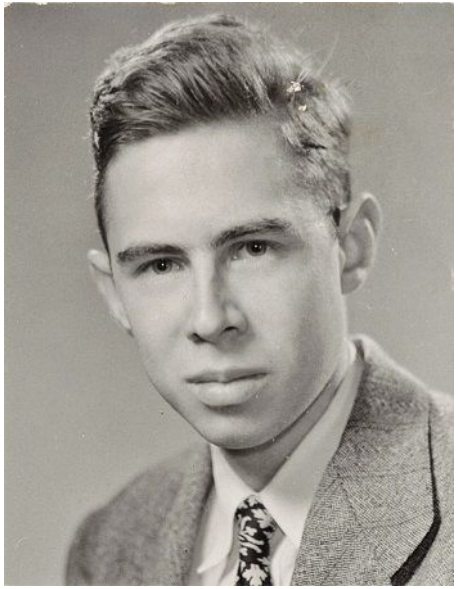
Description : Brain visual pathways. The signals related to the shape, color and motion of visual objects are integrated by specialized brain neuronal populations residing in the primary visual cortex i.e the superficial neuronal layers of the brain occipital lobe. Output signals are then generated that instruct other cortical areas for higher order integration tasks. The " What " pathway connects the primary visual cortex to areas of the temporal cortex that are essential to the recognition and memorization of objects and forms. The " Where " pathway connects the primary visual cortex to areas of the parietal cortex that support perception of precise localization. The interconnections between higher-order visual areas (dashed arrows) as well as other brain areas not highlighted here, allows a fully-integrated perception that takes into account the " What", " Where ", " How " and " When " features of a visual object.

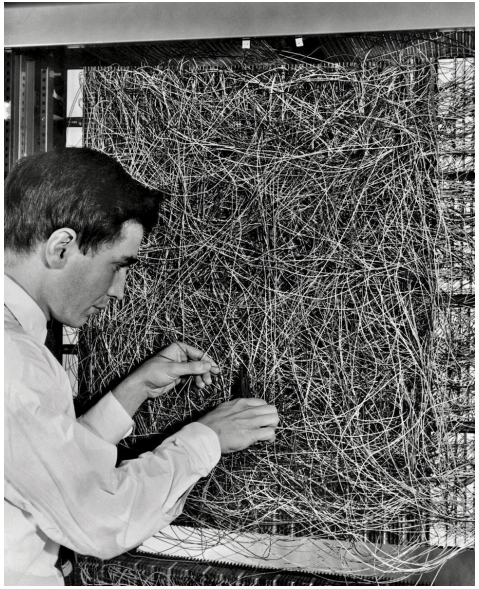
A brief history of deep learning

The first neural network was actually just a neuron, called the **perceptron**.

It was invented in 1957 by **Frank Rosenblatt**.

He designed the Mark I Perceptron, a computer able to learn a model to recognise visual shapes by **adjusting the weights** of a perceptron.

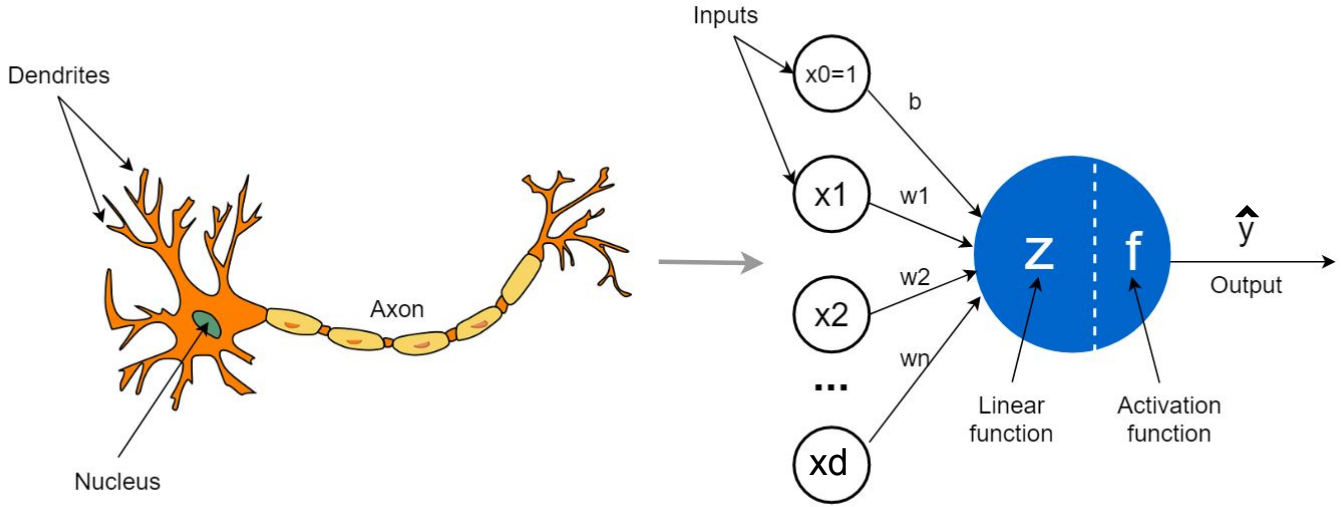




Outline for today

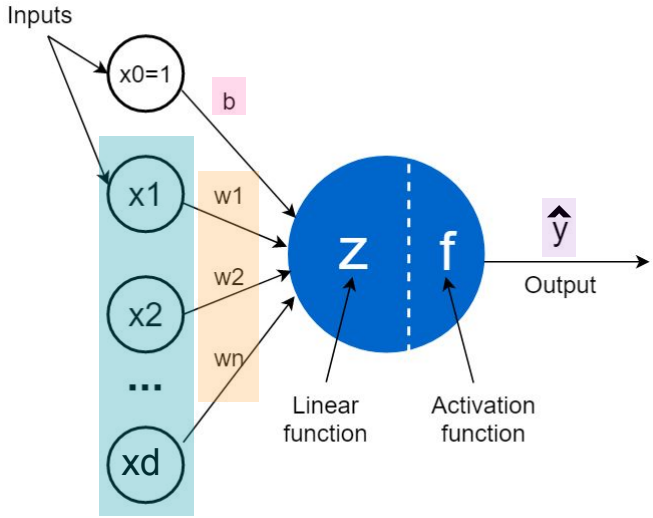
01. Recap of last session
02. A brief history on deep learning for images
03. Our first perceptron
04. Our first multi-layer perceptron
05. Our first convolutional neural network
06. Quiz

The perceptron



Source : https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/

The maths behind the perceptron



Let $\boldsymbol{x} = (x_1, x_2, \ldots, x_d)^\top \in \mathbb{R}^d$ be an input vector,
and $\boldsymbol{w} = (w_1, w_2, \ldots, w_d)^\top \in \mathbb{R}^d$ be a weight vector.

The perceptron computes:

$$z = \boldsymbol{w}^\top \boldsymbol{x} + b$$

where $b \in \mathbb{R}$ is the bias.

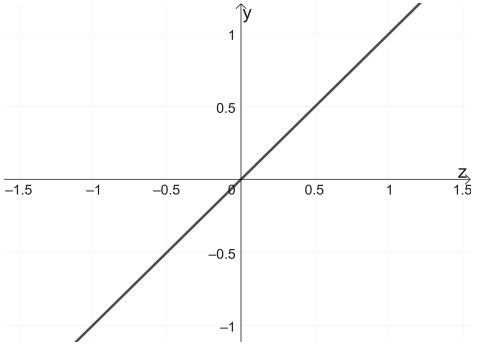
The output is:

$$\hat{y} = f(z)$$

where f is an activation function.

Source : https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/

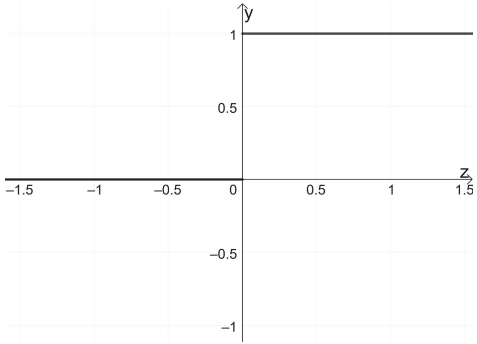
Activation functions for perceptrons



Linear activation function

Made using Geogebra

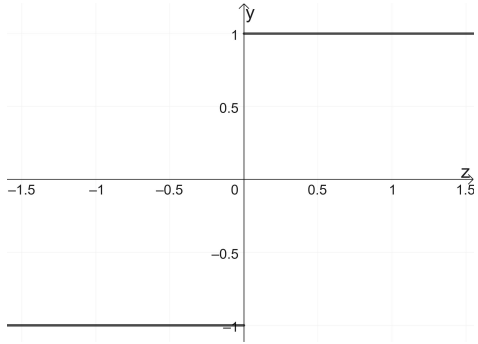
$$\hat{y} = f(z) = z$$



Heaviside step function

Made using Geogebra

$$\hat{y} = f(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases}$$



Sign function

Made using Geogebra

$$\hat{y} = f(z) = \begin{cases} 1 & \text{if } z > 0, \\ -1 & \text{otherwise.} \end{cases}$$

How does a perceptron learn ?

α = learning rate

First : we initialise the weights (randomly is most common).

For *t* in total number of epochs (fixed or based on a rule) :

For *i* in training observations : #We go through the whole dataset

$\hat{y}_i(t) = f(w_0(t)x_{i,0} + w_1(t)x_{i,1} + \cdots + w_n(t)x_{i,n})$ #We compute the prediction

For j in total number of weights : #We update the weights

$w_j(t+1) = w_j(t) + \alpha(y_i - \hat{y}_i(t))$

t = t+1

Interpretation

At each step we update the weights by pulling in the direction that brings us closer to the correct prediction. The furthest away we are from the truth, the stronger is the change.

$$w_j(t+1) = w_j(t) + \alpha(y_i - \hat{y}_i(t))$$

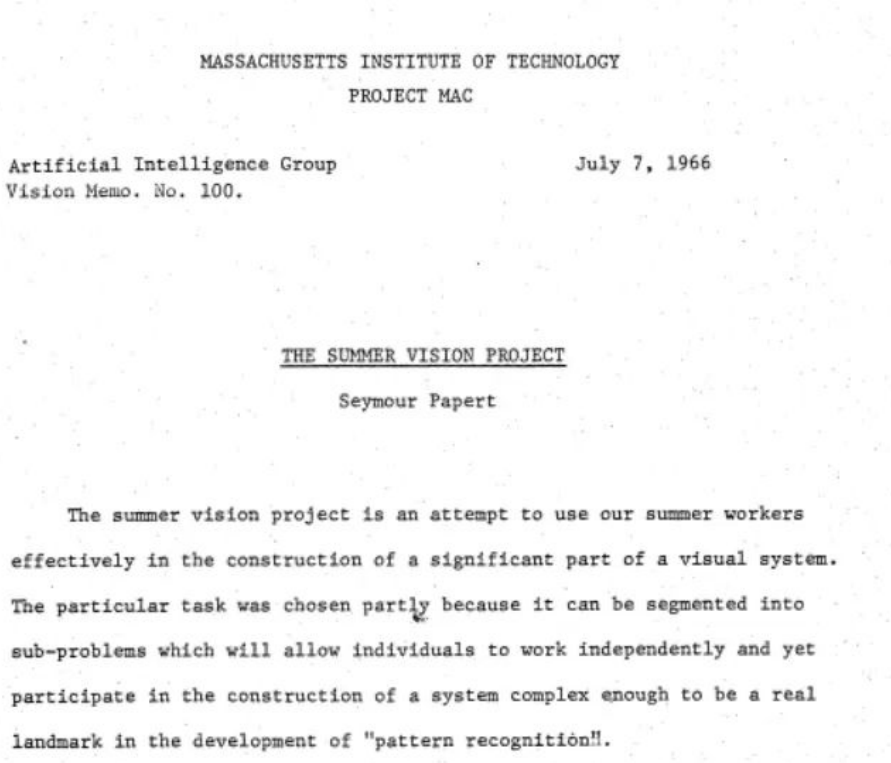
This should theoretically improve performance of the neural network step after step, effectively training the model.

A brief history of deep learning for images

Originally, computer vision didn't seem that "hard".

In 1966, **Seymour Papert**, an MIT professor, aimed to address the challenge of computer vision within a short span of a few months, during what he called the **Summer Vision Project**.

60 years later, computer vision is still a challenge.



MASSACHUSETTS INSTITUTE OF TECHNOLOGY
PROJECT MAC

Artificial Intelligence Group — July 7, 1966
Vision Memo. No. 100.

THE SUMMER VISION PROJECT

Seymour Papert

The summer vision project is an attempt to use our summer workers effectively in the construction of a significant part of a visual system. The particular task was chosen partly because it can be segmented into sub-problems which will allow individuals to work independently and yet participate in the construction of a system complex enough to be a real landmark in the development of "pattern recognition".

Challenges at the time

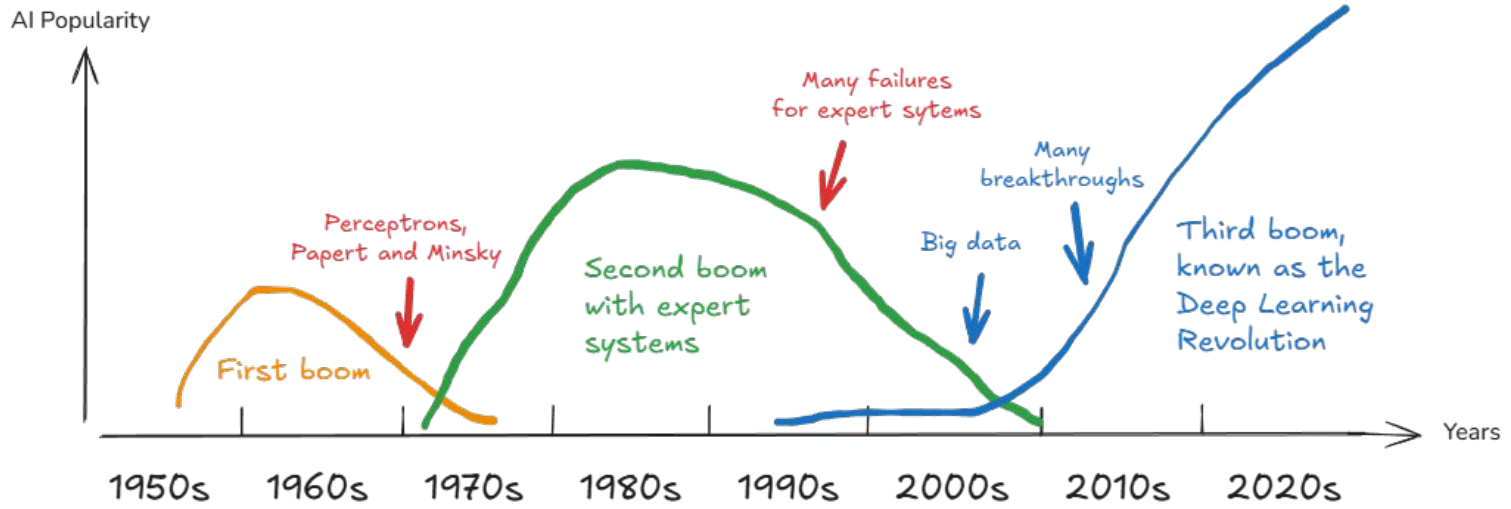
In 1969, Seymour Papert and Marvin Minsky published **Perceptrons**. Their goal was to show Frank Rosenblatt's perceptron strong limitations.

At the time, perceptrons were not that successful because there was :

- not enough data
- not enough computation power

The pessimism of the authors lead to a strong shift in the AI research directions, which lead to the **AI winter** in the 1980s.

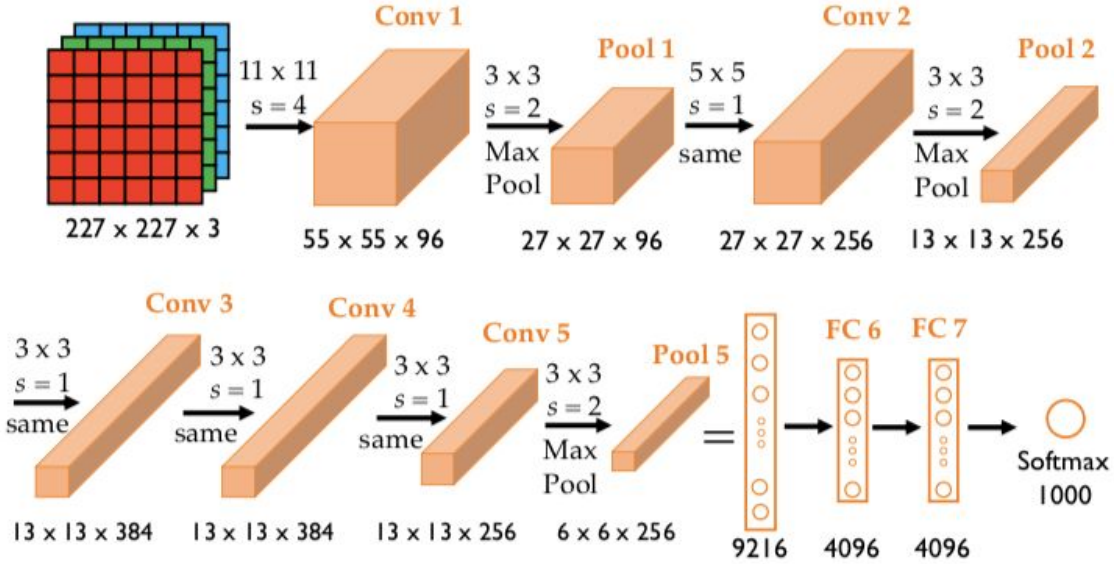
The years passed…



A minimalist timeline of AI popularity over time

Source : Me (Excalidraw)

AlexNet (2012), or the beginning of Deep Learning Revolution



Source : https://medium.com/@saba99/alexnet-f0cb53648600

Wait wait wait… we missed a few steps, right ?

First… what if we decided to connect a few neurons (perceptrons) together ?

Outline for today

01. Recap of last session
02. A brief history on deep learning for images
03. Our first perceptron
04. **Our first multi-layer perceptron**
05. Our first convolutional neural network
06. Quiz

The Multi-Layer Perceptron (MLP)

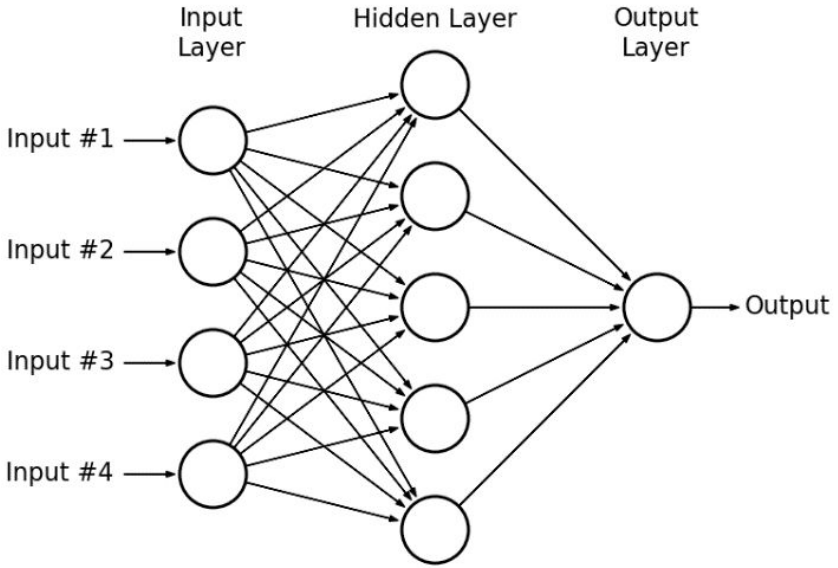
It is possible to add more perceptrons :

Within the same layer:

- To increase the model's capacity to learn multiple patterns in parallel.
- Each neuron optimises its own weights and can extract a different feature.

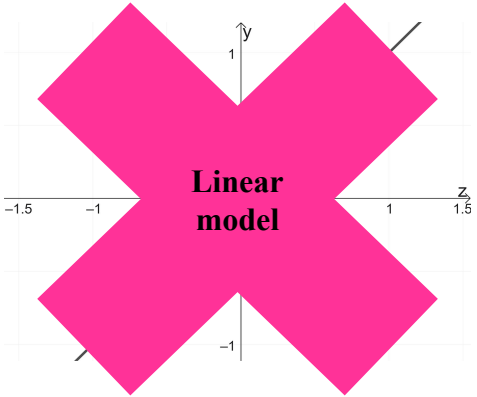
Across multiple layers :

- To stacks several nonlinear transformations.
- To learn increasingly abstract representations, allowing the network to approximate more complex functions.



But there is a trick : Learning a MLP requires differentiable activation functions !

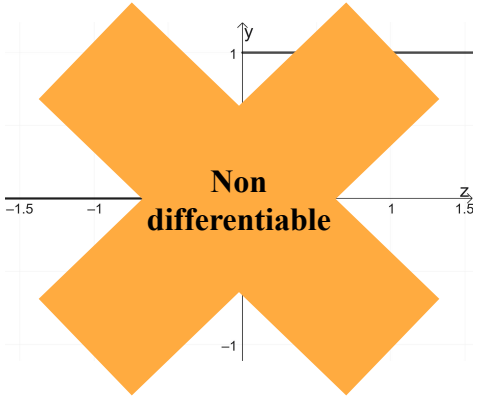
Recall : Activation functions for perceptrons



Linear activation function

Made using Geogebra

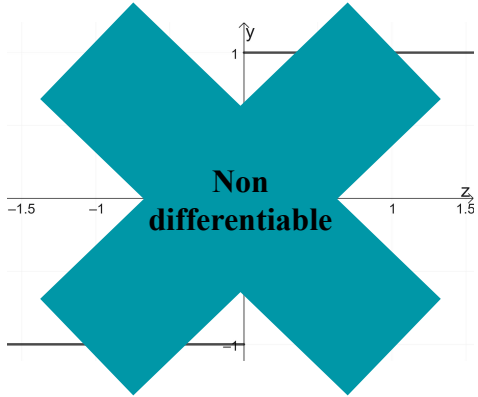
$$\hat{y} = f(z) = z$$



Heaviside step function

Made using Geogebra

$$\hat{y} = f(z) = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases}$$

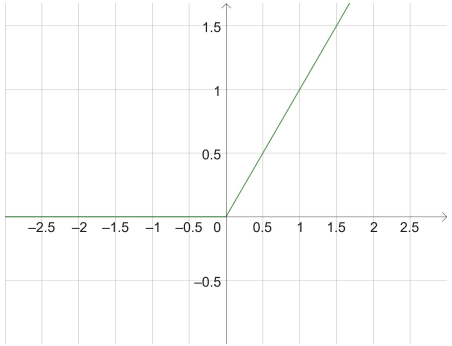


Sign function

Made using Geogebra

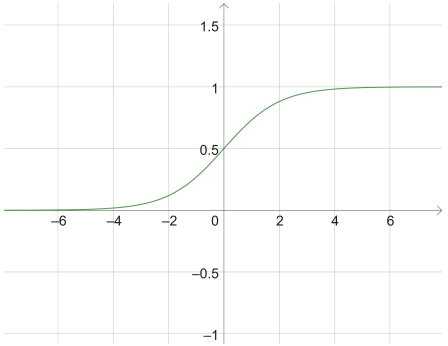
$$\hat{y} = f(z) = \begin{cases} 1 & \text{if } z > 0, \\ -1 & \text{otherwise.} \end{cases}$$

Differentiable activation functions for MLP



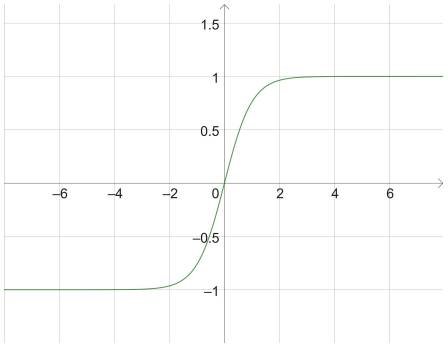
ReLU activation function
Made using Geogebra

$$a = f(z) = \begin{cases} 0 & \text{if } x < 0, \\ x & \text{otherwise.} \end{cases}$$



Sigmoid function
Made using Geogebra

$$a = f(z) = \frac{1}{1 + e^{-z}}$$



Hyperbolic tangent
Made using Geogebra

$$a = f(z) = \tanh(z)$$

Why did you not mention the softmax ?

The softmax is not for hidden layers…

$$\hat{y}_k = \frac{\exp(z_k)}{\sum_{j=1}^{K} \exp(z_j)}.$$

- Softmax forces all activations in a layer to sum to 1, which creates a probability distribution.
- Neurons become highly dependent on each other, so changing one neuron changes all others.
- The gradients often because very small.
- Computation is expensive for every hidden layer.
- We only keep softmax for the output layer !

How does a MLP learn ?

α = learning rate

First : we initialise the weights (randomly is most common).

For *t* in total number of epochs (fixed or based on a rule) :

For *i* in training observations : #We go through the whole dataset

$$\hat{\mathbf{y}} = f^{(L)}\left(\mathbf{W}^{(L)} f^{(L-1)}\left(\mathbf{W}^{(L-1)} \cdots f^{(1)}\left(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}\right) + \mathbf{b}^{(L-1)}\right) + \mathbf{b}^{(L)}\right)$$.#We compute the prediction

For j in total number of weights : #We update the weights

$$w_j(t+1) = w_j(t) - \frac{\partial L(t)}{\partial w_j(t)}$$

t = t+1

Let’s play !

https://playground.tensorflow.org/

The XOR problem

- The XOR (exclusive OR) problem is a **logical operation** that takes two binary inputs and returns true if exactly one of the inputs is true.
- It is a standard application in deep learning, that the perceptron was not able to solve (see playground).
- Example of applications :
  - Detecting anomalies in satellite images : Nasa World View, Night light changes in the Middle East

    Two inputs : “has light” and “is usually inhabited” (people live there).
    - If both are true : ✅
    - If both are false : ✅
    - If is inhabited but has no light : Power outage ⚡
    - If has light but is uninhabited : Intruders detected 🥷

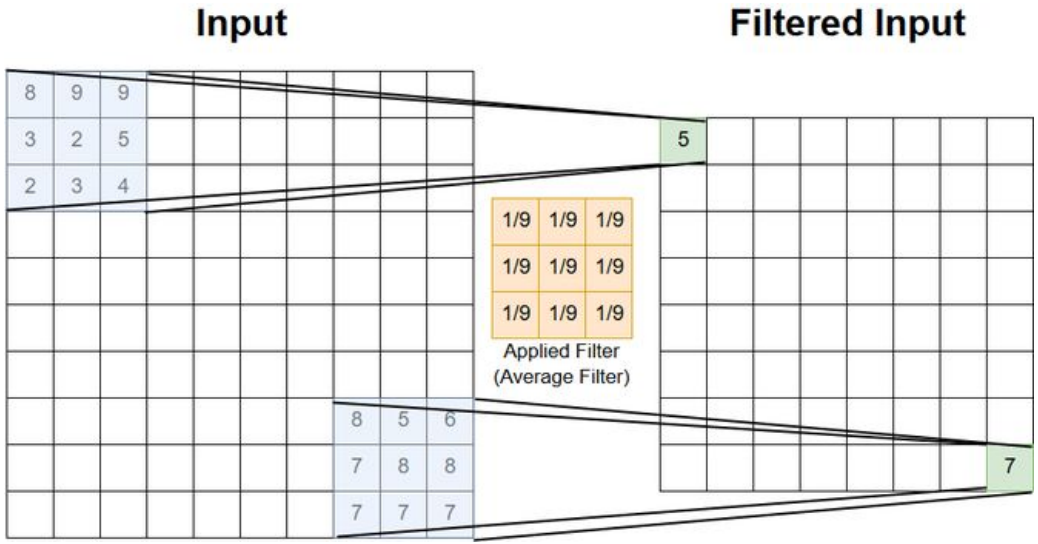
Outline for today

01. Recap of last session
02. A brief history on deep learning for images
03. Our first perceptron
04. Our first multi-layer perceptron
05. **Our first convolutional neural network**
06. Quiz

Convolutional neural networks

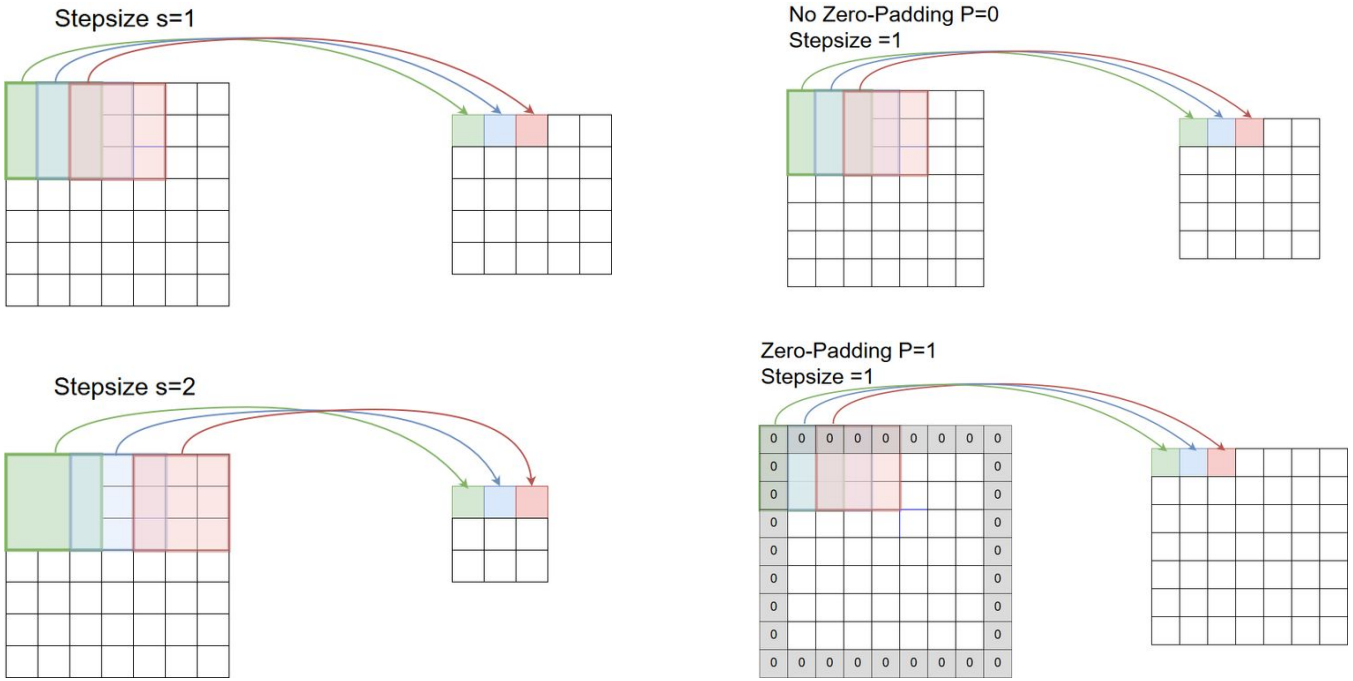
- With MLP, we have the same issue as with traditional machine learning :
  - Giving each pixel as an input yields input vectors of very high dimensions.
  - We cannot easily recognise patterns or shapes if they are in different positions.
- Remember how we applied Gaussian filter using convolutions ?
  It was a good way to “sum up” the information locally !
- That is exactly what we are going to do with Convolutional Neural Networks (CNN)!
- Our model will learn to generate its own features, which are appropriated for the task!
- See convolution examples.

Reminder on the convolution



Source : https://hannibunny.github.io/mlbook/neuralnetworks/03ConvolutionNeuralNetworks.html

Convolution layer parameters

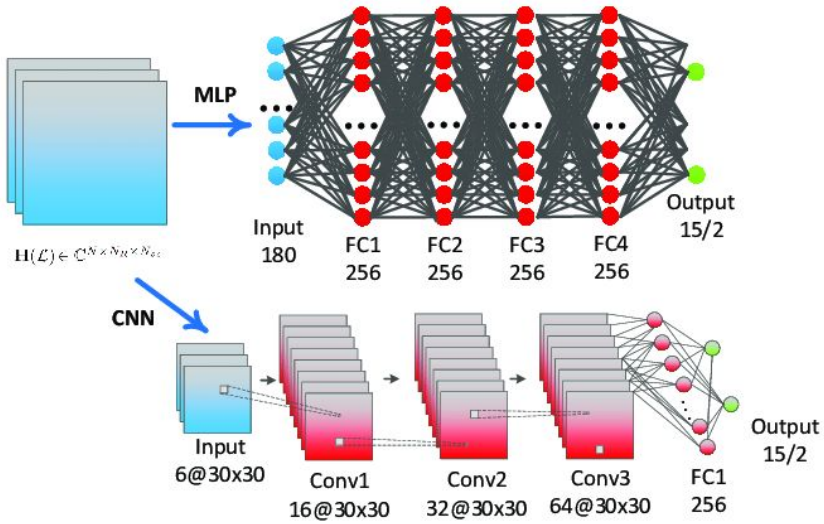


Source : https://hannibunny.github.io/mlbook/neuralnetworks/03ConvolutionNeuralNetworks.html

Advantage of convolutional neural networks

Let's say we are working with 100x100 coloured images.

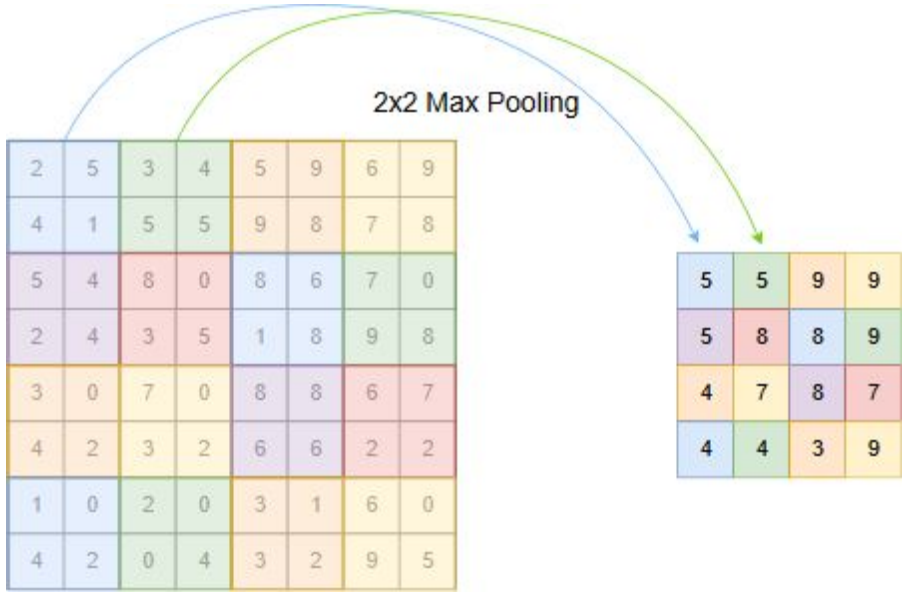
- ➢ One MLP with 100 neurons = 3 000 100 parameters !
- ➢ One CNN with 100 filters = (3x3x3 + 1) x 100 = 2 800 parameters !



Source : Xiang, Chenlu & al. (2019). Robust Sub-Meter Level Indoor Localization With a Single WiFi Access Point-Regression Versus Classification.

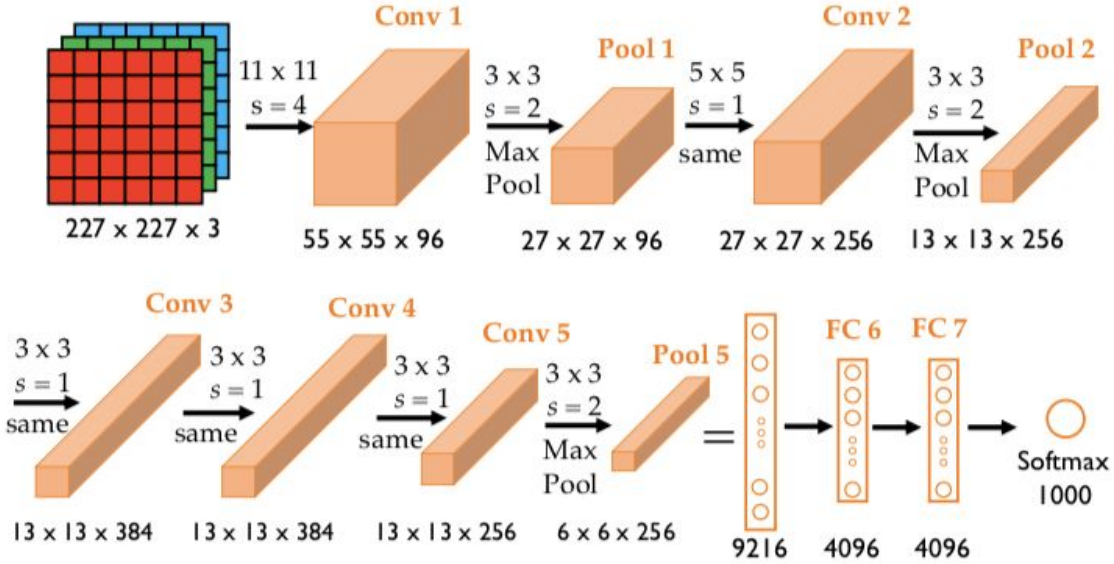
Pooling

- Convolution filters detect local patterns.
- Pooling compresses the information while keeping the most important activations.
- This makes the network more robust and reduces computation.
- Almost all classical CNN (including AlexNet) chain convolutional layers with pooling layers !



Source : https://hannibunny.github.io/mlbook/neuralnetworks/03ConvolutionNeuralNetworks.html

Example for AlexNet



Source : https://medium.com/@saba99/alexnet-f0cb53648600

Outline for today

01. Recap of last session
02. A brief history on deep learning for images
03. Our first perceptron
04. Our first multi-layer perceptron
05. Our first convolutional neural network
06. **Quiz**

Quiz 1

- Take a pen and a piece of paper.
- Write your full name and the date.
- Answer the questions in 10 minutes.
- Wait 5 extra minutes without moving.

Question 1 : Let us assume we are working with a coloured image of size 200x200. What does the computer see ? How many pixels are there in total ?

Question 2 : One might extract features from images before applying a machine learning model on the features instead of on the raw pixels. Give one example of a feature that can be extracted using standard image processing techniques. Explain, in one or two sentences and with your own words, how this feature can be obtained.

See you this afternoon !